SKJS Type Checker Architecture and Type System

SKJS Documentation Team

November 13, 2025

Abstract

Static typing for JavaScript must repeatedly trade precision for soundness in the presence of union types, generics, and asynchronous control flow. This paper presents *SKJS*, a sound type system that prioritizes safety while remaining usable on idiomatic JS/TS code. Our contributions are fourfold: (1) we formalize union normalization in negative (argument) positions and show that SKJS rejects exactly the unsound calls while permitting a large class via domain normalization; (2) we explain the conservative treatment of nested `Promise` values (no implicit flattening) and show that generic higher-order wrappers that try to unify incompatible domains must propagate `never`; (3) we enforce contravariant method subtyping, eliminating TypeScript-style method bivariance; and (4) we evaluate expressivity on representative real-world idioms (routing/middleware, event handlers, and result-typed APIs), distilling practical workarounds (local guards, adapter shims, and small facades for ambient libraries). The key finding is that SKJS is *conservative but sound*: function unions are callable only when their domains coincide; generic combinators that would otherwise admit unsound calls collapse to `never`; and branch-based refinement cannot resurrect information lost to contravariance. We additionally catalogue expressivity gaps such as layout-divergent (union-of-record) types with correlated fields and offer low-cost patterns that recover the intended programs with predictable ergonomics. The accompanying examples are runnable, serve as regression tests, and clarify that alleged "argument-position unsoundness" arises from an intentionally under-approximating normalizer designed to keep the calculus sound while remaining practical.

Contents

1 Introduction **4**

2 Formal Preliminaries **4**
- 2.1 Notation and Mathematical Background . . . 4
- 2.2 Type Language: Syntax and Well-Formedness . . . 4
- 2.3 Semantic Interpretation with Environments . . . 4
- 2.4 Subtyping and Variance . . . 4
- 2.5 Normalization to DNF . . . 5

3 Repository Composition and Build Process **6**
- 3.1 Workspace Layout . . . 6
- 3.2 Source Module Overview . . . 7
- 3.3 Skip Build . . . 7
- 3.4 CLI Workflow . . . 8
- 3.5 Binary Distribution . . . 8

4 Core Type Representation **8**
4.1 Disjunctive Normal Form and Type Objects . . . 8
4.2 Clauses, STypes, and Instances . . . 9
4.3 Primitive, Literal, and Composite Types . . . 9
4.4 Type Parameters and Environments . . . 9

5 Checker Pipeline **10**
5.1 From Files to Definitions . . . 10
5.2 Incremental Checking and Caching . . . 10
5.3 Error Reporting and Diagnostics . . . 10

6 Subtyping, Unions, and Constraint Solving **10**
6.1 Clause-Level Subtyping . . . 10
6.2 Algebraic Data Types and Objects . . . 11
6.3 Function Variance . . . 12
6.4 Constraint Solving via `TEnv` . . . 12

7 Flow-Sensitive Refinement and Scopes **12**
7.1 Scope Model . . . 12
7.2 Condition Evaluation . . . 13
7.3 Loop and Switch Handling . . . 13

8 Expression Checking Algorithms **14**
8.1 Literal and Reference Evaluation . . . 14
8.2 Object and Algebraic Construction . . . 14
8.3 Calls, Constructors, and Templates . . . 14
8.4 Assignments and Local Object Growth . . . 15
8.5 In-Operator Semantics . . . 15

9 Statements, Functions, and Classes **16**
9.1 Statements . . . 16
9.2 Pattern Binding Walkthrough . . . 16
9.3 Function Checking . . . 17
9.4 Class Checking . . . 17

10 Stubs and Testing **18**
10.1 Import Stubs and Ambient Modules . . . 18
10.2 Examples and Expect-Error Suites . . . 18
10.3 Conformance Harness . . . 18

11 Module Reference Summary **19**

12 Surface Language Constraints and Future Work **19**
12.1 Current Limitations . . . 19
12.2 Future Enhancements . . . 20

13 Illustrative Flow Derivation **21**

14 Case Study: Refinement Workflow **21**

15 Union Normal Form Proof Overview **22**

16 Algebra of Unions (laws used in code) **28**

17 Expressivity, Promise Semantics, and Framework Interoperability **29**
17.1 Bivariance in TypeScript Frameworks 33
17.2 Background: where bivariance comes from in TS 33
17.3 Survey of common Node/TS back-end APIs (actual code) 34
17.4 Will bivariance in frameworks block interop? 35
17.5 Binding patterns (no type-system changes) 35
17.6 How important are the expressivity gaps? 37
17.7 Additional TypeScript testlets (bivariance and bindings) 38
17.8 Conclusions & examples to keep 38

18 Testing and Behavioral Evidence **39**

19 Extensions **40**
19.1 Per-ADT Discriminator Names 40

1 Introduction

The SKJS repository hosts a Skip-based implementation of a sound subset of the TypeScript language. The project integrates a Skip frontend and an extensive suite of examples and negative tests to ensure that the checker both enforces the intended restrictions and preserves developer ergonomics. This report surveys the repository structure, build process, type representation, subtyping and constraint solving, checking algorithms for expressions, statements, classes, and functions, and the testing ecosystem. The analysis is grounded directly in the Skip sources under `src/`, the language rules captured in `skjs-rules.md`, the build manifest `Skargo.toml`, build script `build.sk`, binary distribution guide `docs/binary-distribution.md`, and the examples and test suites under `examples/` and `tests/`. Inline citations point to the authoritative files (e.g., "see `src/TypeCheckExpr.sk`").

2 Formal Preliminaries

2.1 Notation and Mathematical Background

We write $\mathcal{P}(X)$ for the powerset of X, $\times$ for Cartesian product, and $\subseteq$ for subset. A complete lattice $(L, \leq)$ admits arbitrary joins/meets; we write $\bigsqcup$ and $\bigsqcap$. If $F : L \to L$ is monotone, the least fixed point μF exists (Tarski) and can be computed as the limit of the increasing chain $\bot \leq F(\bot) \leq F^2(\bot) \leq \ldots$ (Kleene). We use $[\![\cdot]\!]_\rho$ for denotation under type-parameter valuation ρ, $\top/\bot$ for top/bottom, and $\uplus/\sqcup$ for clause/clauses unions.

2.2 Type Language: Syntax and Well-Formedness

We use the abstract syntax in Section 2.2 (restated here succinctly): single-type constructors (`SType`) include units, literal primitives, tuples, promises, objects (fields and at most one index), algebraic data types (ADT branches keyed by a distinguished `kind`), and sets of functions (overloads). General types (`TType`) combine an `Insts` set (variables, named instances, deferred union/intersection) with a DNF of `Clauses`. Well-formedness invariants include: unique clause per quantified-variable set; top-canonicalization (single top clause); omission of bottom clauses; disjointness of ADTs and objects; and no instances when `Clauses`$= \top$.

2.3 Semantic Interpretation with Environments

Let ρ map type parameters to subsets of values. We interpret `TVar` as $\rho(X)$; `TSInst` via its (possibly recursive) alias equation as μF; `TUnion`/`TInter` as set union/intersection; tuples as products; promises covariantly in their payload; objects covariantly by fields and index signatures; ADTs as tagged unions; functions contravariantly in parameters and covariantly in returns. Full definitions appear in Definition 1.

2.4 Subtyping and Variance

We assume standard subtyping: units and primitives form flat hierarchies; tuples are pointwise; objects are width/depth-subtyped by fields (with optionality), index signatures constrain key/value types; ADTs subtype per-tag; promises are covariant; functions are contravariant in parameters and covariant in returns. Overloads are matched by arity and parameter shapes. Importantly, `TFuns::union` computes a greatest lower bound under subtyping (intersecting domains, unioning codomains), not a set-theoretic union of behaviours.

Definition (Positivity and well-formedness).

We write X *positive* in a type T if it appears only covariantly according to the variance algebra:

- In $A \to B$, occurrences in A are *negative* (contravariant) and in B are *positive*.
- In tuples, promises, arrays, and read-only object fields, occurrences are positive component-wise.
- In intersections $T_1 \wedge T_2$ and unions $T_1 \mid T_2$, positivity is inherited in both operands.
- In mutable object fields and index-signature ranges used for writes, occurrences are *invariant*; recursive occurrences under such positions are disallowed.

A binding $\mu X.T$ or $\forall X.T$ is *well formed* only if X is positive in T. This ensures the corresponding semantic operator is monotone, so least fixed points exist (Tarski) and are constructible by Kleene iteration over our normal forms.

Note (TypeScript methods). TypeScript historically treats method parameters bivariantly; SKJS uses true contravariance. All examples below use arrow functions to avoid conflating this difference.

READ UP TO HERE

2.5 Normalization to DNF

Clauses and clause-maps (`Clauses`) maintain canonical DNF with invariants enforced by constructors (top collapse, bottom omission, unique key). Normalization distributes intersections over unions and rekeys by quantified-variable sets. Lemma 0.a and Lemma 0.b (later) state denotation preservation and normal-form maintenance. A small example:

$$\forall X.(S_1 \mid S_2) \ \wedge \ \forall X.S_3 \equiv \forall X.\big((S_1 \wedge S_3) \ \mid \ (S_2 \wedge S_3)\big)$$

provided that both quantifiers range over the same α-normalized binder set (no capture), and X occurs only *positively* in S_1, S_2, S_3 (see Def. 2.4).

```
type Forall1 = <X>(x: X) => X;
type Forall2 = <Y>(y: Y) => Y;
// The union Forall1 | Forall2 still tracks the distinct binder names.
// Treating the union as a single forall would be unsound.
```

We formalize the core operators:

- Clause union $c_1 \uplus c_2$ requires equal quantified-variable sets and unions their `STypes` payloads (or yields top if either is top). Bottom payloads are omitted.
- Clauses union $C_1 \sqcup C_2$ is key-wise: missing keys are kept; shared keys use $c_1 \uplus c_2$; a top result collapses the map to a single top clause.

Both preserve denotations and canonical form (proved later as Lemma 0.a/0.b).

TypeScript Examples.

```typescript
// Unit and literal primitive singletons
const emptyPort: null = null;
// EXPECT-ERROR: Unnecessary 'as const'
const literalAnswer = 42 as const;

// Tuple constructor
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const pair: [number, string] = [42, "engineers"];

// Promise payloads
async function fetchFlag(): Promise<boolean> {
  return true;
}

// Object with optional field and index signature
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const cache: { hits?: number; [key: string]: number | undefined } = {
  hits: 1,
  miss: undefined,
};

// Algebraic data type (ADT) branch keyed by `kind`
type Result<T> =
  | { kind: "success"; data: T }
  | { kind: "error"; message: string };
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const ok: Result<number> = { kind: "success", data: 1 };

// Function overload set (represented as TFuns)
function length(value: string): number;
function length<T>(value: T[]): number;
function length(value: string | unknown[]): number {
  return value.length;
}

// Clause-level union merges compatible payloads
type WithFlag = { flag: true } | { flag: false; reason: string };
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const denied: WithFlag = { flag: false, reason: "off" };
```

3 Repository Composition and Build Process

The preceding Formal Preliminaries introduce the mathematical model; we now pivot to an implementation-oriented tour of the codebase that realizes those definitions. Subsequent sections reference the same notation without re-deriving it.

3.1 Workspace Layout

The Skip crate is declared in `Skargo.toml`, which names the `skjs` package, points to the upstream Skip standard library and CLI dependencies, and registers a binary target whose entry point is

`SKJS.main` (`Skargo.toml`). Skip sources live under `src/`, where parser, environment, type, and checker modules are grouped by responsibility. Supporting assets include:

- `skjs-rules.md`: documentation of the supported TypeScript surface and known limitations.
- `docs/binary-distribution.md`: CircleCI-based release process.
- `examples/src/*.ts`: curated positive programs showcasing supported features.
- `tests/expect-error`: negative cases that must be rejected.
- `tests/ts-conformance`: harness for importing upstream TypeScript suites via `devtools/copy-ts-tests.sh`.
- `import-stubs/`: ambient module definitions (Fastify, Pino, `@types`, etc.) resolved during checking.

3.2 Source Module Overview

The Skip sources under `src/` are grouped logically:

- Frontend: `Lexer.sk`, `Parser*.sk`, and `Ast.sk` build the TypeScript subset AST and track source positions via `Position.sk` and `NewLines.sk`.
- Dependency graph and storage: `SKStoreUtils.sk`, `ModuleResolver.sk`, `Files.sk`, `FileDefs.sk`, and `ClassHeriarchy.sk` control how files are loaded, parsed, cached, and related (e.g., inheritance graphs used by class checking).
- Type machinery: `TType*.sk` (representation, unions, intersections, transformers, substitution, pretty-printing) plus `TEnv.sk` (normalization and instantiation).
- Checking pipeline: `TypeCheck.sk` declares the main `TypeChecker` class, while `TypeCheckExpr.sk`, `TypeCheckStmt.sk`, `TypeCheckClass.sk`, `TypeCheckFun.sk`, `TypeCheckScope.sk`, `TypeCheckEnv.sk`, `TypeCheckRefine.sk`, `TypeCheckUtils.sk`, and `TypeCheckDef.sk` (top-level definitions) provide specialized routines. `TypeCheckDef.sk` wires parsing results into checker invocations and also enforces project-wide constraints (e.g., allowed declaration kinds).
- Diagnostics: `TypingError.sk` formats multi-position errors, while `Reachability.sk` and `Termination.sk` compute control-flow summaries used to detect unreachable code and missing returns.

Understanding these modules clarifies where each concern lives when extending the checker.

3.3 Skip Build

The build pipeline is configured by `build.sk`, which tracks files for incremental rebuilds, configures position-independent code according to `RELOCATION_MODEL`, and injects release or debug flags.

3.4 CLI Workflow

`src/Main.sk` exposes two primary subcommands implemented with `skiplang/cli`: `check` (type-check files) and `run` (type-check and then execute with Bun). The CLI wires reusable arguments such as `--dir`, `--base-url`, `--import-stubs`, and persistent cache knobs (`--init`, `--data`) so teams can amortize parse and type information across runs. The `run` subcommand reuses `execCheckWithStatus` to ensure the program is type safe before delegating to Bun via `Posix.execvp`. Verbose and backtrace flags toggle additional logging. `check` defaults to scanning `src/**/*.ts` but also accepts explicit file lists, aligning the CLI with both CI pipelines and editor integrations.

3.5 Binary Distribution

Continuous delivery relies on CircleCI, as documented in `docs/binary-distribution.md`. Two jobs—`build_and_test` and `build_binaries`—validate the code and produce platform-specific artifacts named `skjs_{COMMIT}_{PLATFORM}_{ARCH}`. Artifacts are retained for 30 days and accessed via authenticated API calls or the convenience script `scripts/fetch-binary.sh`. GitHub Actions is reserved for presubmit testing, while CircleCI handles signed releases and metadata management. Local developers can invoke `./scripts/build-release.sh` or run `skargo build --release` (after cloning the Skip toolchain) to build the CLI offline. The fetch script supports selecting commits or branches, configuring output directories, skipping tests for faster iteration, and authenticating via the `CIRCLECI_TOKEN` environment variable. By keeping binaries off-repo, SKJS avoids churn in source control while still offering deterministic builds for downstream consumers who need to integrate the checker into larger Skip programs.

4 Core Type Representation

This section complements the formal syntax of Section 2.2 with implementation details from `src/TType*.sk`. We refer back to the formal definitions rather than restating them.

4.1 Disjunctive Normal Form and Type Objects

The central type representation is `TType` (`src/TType.sk`). Each `TType` stores:

1. A position `pos` used for diagnostic attribution.
2. A cached hash for structural deduplication.
3. `insts`: an ordered set of `TInsts` (type variables, unexpanded named instances, and deferred intersections).
4. `clauses`: a disjunctive normal form (DNF) of conjunctions—called clauses—that mix universally quantified parameters and single-type payloads (`Clauses` and `STypes`).

The interpretation is $[\![t]\!] = [\![t.\text{insts}]\!] \sqcup [\![t.\text{clauses}]\!]$. Constructors such as `TType::bottom`, `TType::top`, `TType::single`, and smart constructors for variables and class instantiations enforce canonical hashing and DNF invariants.

4.2 Clauses, STypes, and Instances

`Clauses` capture unions-of-intersections; each clause holds a set of universally quantified variables and an optional `STypes` payload. `STypes` lift single-type terms such as primitives (`TPrimValue`), units (`TPrimUnit`), tuples (`TTuple`), algebraic data types (`TAlgebraic`), promises, and function sets (`TFuns`). Instances include:

- `TVar`: existential placeholders produced during inference.
- `TSInst`: named generics awaiting expansion.
- `TInter`/`TUnion`: deferred intersections or unions introduced to break cycles (`src/TTypeInter.sk`, `src/TTypeUnion.sk`).

Helper operations like `mapSTypes`, `mapClauses`, and `mapTParams` model variance-sensitive rewrites that rebuild union/intersection structures through the current substitution and typing environment. This is central to flow-sensitive updates, where terms are filtered or augmented without losing memoized invariants.

4.3 Primitive, Literal, and Composite Types

Primitive domains (string, number, bigint, boolean) are represented by `TPrimValue`, which tracks both the kind and an optional set of literal values. Units such as `null`, `undefined`, and `void` use `TPrimUnit`. Tuples are modeled via `TTuple`, arrays are thin wrappers over tuples (empty literal arrays become tuples, as enforced by `skjs-rules.md`), and object literals become `TObject` instances with named and index fields. Algebraic data types (ADTs) are `TAlgebraic` values, i.e., tagged unions whose branches are constrained by a dedicated `kind` field (`TEnv.createTFields` in `src/TEnv.sk`). Function sets are `TFuns`, which keep distinct tables for "normal" functions and "methods" to respect method receiver rules.

4.4 Type Parameters and Environments

`TEnv` (`src/TEnv.sk`) glues AST-level declarations to the type runtime. Its responsibilities include:

- Tracking type parameters in scope (`tparams`) and the instantiation-time substitution map.
- Ensuring only a single expansion level for recursive types (`seenInstantiate`) and guarding against cyclic unions/intersections (`seenUnion`, `seenInter`).
- Normalizing AST type expressions (`normalize`), instantiating class bodies (`new`, `newObject`, `newConstructor`), and enforcing type-parameter constraints via subtyping tests.
- Generating implicit type arguments when a generic class is constructed without explicit parameters (`makeTArgs`).
- Providing structural introspection for keys (`keys`, `forInKeys`) and field creation (`createTFields`), which is reused both by type declarations and literal expressions.

The environment is referenced by every checking pass. For example, resolving `new C<T>` constructs uses `TEnv.new` to produce an object type that incorporates field constraints declared in the class definition. Likewise, flow refinements rely on `TEnv.keys` to interpret `in`-checks on objects.

Both `TType::expand` and `TType::getSTypes` rely on `TEnv` to decide when to inline instances. Expansions are variance-aware: covariant queries distribute unions and intersections differently

than contravariant queries, and the mapper functions in `TTypeTransformers.sk` encapsulate this knowledge. Consequently, every time a clause is inspected, the checker knows whether it is safe to grow or shrink the type without violating previous constraints.

5 Checker Pipeline

5.1 From Files to Definitions

`TypeCheckDef.sk` iterates over parsed AST definitions, dispatching to `TypeChecker::class_`, `TypeChecker::funDef`, or expression-level entry points depending on the declaration kind. The module collaborates with `FileDefs.sk` and `ModuleResolver.sk` to ensure that import graphs are resolved before checking begins. Each file is associated with a `CGNode` so that function dependencies can be tracked during analysis and reused for incremental builds.

5.2 Incremental Checking and Caching

The combination of `SKStoreUtils.sk` and `SKStore.Context` (referenced throughout `TEnv.sk` and `TypeCheckEnv.sk`) enables memoization of parsed files, type normalizations, and call graph relationships. When `recordCalls` is invoked inside `TypeCheckExpr.sk`, the edge is stored in the context so that subsequent runs can reuse the graph unless the callees changed. Similarly, `TEnv.defMap` provides AST snapshots keyed by definition names, so once a type alias or class has been normalized it can be re-instantiated quickly.

5.3 Error Reporting and Diagnostics

`TypingError.sk` accepts arrays of `(PosRange, message)` pairs. Each checking routine accumulates context before calling `typingError`, ensuring that users receive both the primary failure location and any relevant witnesses (e.g., "expected type declared here"). The pretty-printer in `TTypePp.sk` produces human-readable strings for inclusion in these errors. Additionally, `Reachability.sk` and `Termination.sk` annotate AST nodes with control-flow summaries so that diagnostics such as "Missing return statement" or "Unreachable code" can quote exact statement spans.

6 Subtyping, Unions, and Constraint Solving

The implementation refines the abstract subtyping rules introduced earlier. The material below focuses on algorithmic structure; refer to the Formal Preliminaries for semantic contracts.

6.1 Clause-Level Subtyping

`src/TTypeSubtyping.sk` encapsulates subtyping. The algorithm recursively descends through clause combinations. Each clause from the left-hand side must be subsumed by some clause on the right-hand side, provided they agree on universally quantified variables. When both clauses contain `STypes`, the check delegates to `STypes::isSub`, which aligns single-type lists and invokes `isSubSType` per entry. Errors such as `STTooManySType`, `STMissingField`, and `STTooManyTFun` pinpoint exact mismatches for diagnostics emitted by `TypeChecker::checkSubType` (`src/TypeCheckUtils.sk`).

TypeScript Example.

```ts
type Animal = { kind: "animal"; legs: number };
type Dog = Animal & { bark(): void };

declare const spot: Dog;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const asAnimal: Animal = spot;
// Dog <: Animal because every clause of Dog refines Animal

type Nullable<T> = T | null;
declare const maybeDog: Nullable<Dog>;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const maybeAnimal: Nullable<Animal> = maybeDog;
// Union clauses checked individually
```

6.2 Algebraic Data Types and Objects

ADTs leverage `TAlgebraic::isSub` to compare branch sets: each branch in the subtype must appear in the supertype with compatible constructor fields. For objects, `TFields::isSub` checks both explicit fields and index signatures. Optional fields are widened by unioning `undefined` before comparison, mirroring how SKJS treats TypeScript optional properties (`src/TTypeSubtyping.sk`). Index signatures are validated by first ensuring key types align and then recursively checking value types; lacking optionality triggers `STMissingField`.

TypeScript Examples.

```ts
// ADT subtyping
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; size: number };
type ColoredShape =
  | { kind: "circle"; radius: number; color: string }
  | { kind: "square"; size: number; color: string };

declare const palette: ColoredShape;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const plain: Shape = palette; // Each branch adds extra fields that subtype covariantly

// Object subtyping with optional fields
type Config = { host: string; port?: number };
type DerivedConfig = { host: string; port?: number; protocol: "http" | "https" };
declare const derived: DerivedConfig;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const base: Config = derived;

// Index signature compatibility
type Cache = { [key: string]: number };
type ExtendedCache = { [key: string]: number | undefined; hits?: number };
declare const extended: ExtendedCache;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const cache: Cache = extended; // `undefined` is unioned into the value type
```

6.3 Function Variance

`TFuns::isSub` ensures that, for each overload in the left operand, there is an appropriately typed overload on the right. Parameters are contravariant and return types covariant, implemented by `checkSubType` calls inside `TypeChecker::call` (`src/TypeCheckExpr.sk`) and `TFuns::isSub` (`src/TTypeSubtyping.sk`). Methods and free functions are stored separately to prevent mixing receiver expectations.

TypeScript Example.

```
type Handler = (input: string) => { ok: true; value: string };
type WiderHandler = (input: string | number) => { ok: true; value: string };

declare const h: Handler;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const acceptWider: WiderHandler = (input) => ({ ok: true, value: String(input) });

// Contravariance on parameters, covariance on returns
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const delegate: Handler = acceptWider; // OK: accepts a supertype of 'string'
```

6.4 Constraint Solving via `TEnv`

Constraint solving spans multiple modules:

- `TTypeSubst.sk` records substitution state. Every time a `TVar` is instantiated, it updates `subst.version`, signalling dependent routines (e.g., `TypeChecker::call`) to re-check arguments.
- `TTypeTransformers.sk` contains generic mappers that rebuild types under variance annotations. These are leveraged when substituting type parameters and when narrowing locals so that variance is respected.
- `TTypePp.sk` pretty-prints types for diagnostics; while not part of solving per se, it exposes enough structure that debugging constraint failures becomes tractable.

The aggregate effect is a lazy, memoized solver that can express recursive constraints (through unexpanded instances) while still providing precise errors. Type parameter constraints (`extends` clauses) are enforced during `TEnv.instantiate`: each argument is normalized and checked against the declared constraint using `isSubTType`. During function calls, `TypeChecker::call` synthesizes fresh type variables if explicit arguments are omitted, then replays parameter checking until the substitution stabilizes (tracked via `subst.version`). Intersections inject constraints into `TVar` objects by calling `TVar::addConstraint` when one side reduces to a single variable (`src/TTypeInter.sk`). This design cleanly separates symbolic constraint accumulation from concrete clause expansion.

7 Flow-Sensitive Refinement and Scopes

7.1 Scope Model

The mutable environment `TCEnv` (`src/TypeCheckEnv.sk`) bundles the global substitution, current `Scope`, call graph context, and a map of "local objects" that track object literals bound to locals so that their types can grow until the value escapes. Each `Scope` maintains local bindings and

references, supports hierarchical lookup, and records whether a variable is accessed through a loop to guard against unsound back edges (`src/TypeCheckScope.sk`). Scope merging operations merge or replace bindings when branching structures rejoin, using union to approximate types along both control-flow paths. Internally, `Scope::escapingRefs` enumerates bindings that flow out of the scope (i.e., locals defined in an inner block but referenced by outer scopes). `Scope::popMerge` unions these with the parent scope, while `Scope::popReplace` overwrites parent entries when both branches agreed on the same binding. This fine-grained control avoids needless widening. Meanwhile, `TCEnv.localObjects` stores either `LOCanGrow(bound)` or `LOUsed(pos)`, allowing the checker to detect when an object literal should stop accumulating fields (e.g., after it is passed to a function call).

7.2 Condition Evaluation

`TypeCheckRefine.sk` defines `TypeChecker::evalCond`, which interprets boolean expressions and refines locals accordingly. Supported refinements include:

- Strict equality/inequality against literal strings, `null`, or `undefined`.
- Algebraic branch discrimination via the special `kind` field (`evalAlgebraicBranchEquality`), which filters `TAlgebraic` branches.
- `typeof` checks that isolate primitive kinds and optionally inject default witnesses (e.g., string literal "ready") when narrowing to a singleton type.
- `instanceof` checks that replace the local type with the instantiated class type computed by `TEnv.new`.
- `in` checks that either add a field to the object type (when the branch succeeds) or remove it on the false branch, using `getInLhsTypeAndFields` from `src/TypeCheckExpr.sk`.

Refinements rewrite the local's `TType` via `setLocal`, which expands the current type, filters its `STypes` with predicates, optionally injects default literals, and writes the result back into the current scope. All refinements operate on locals; comparing property accesses directly triggers the "Only locals can be compared to null/undefined" error documented in `skjs-rules.md`.

7.3 Loop and Switch Handling

Loops rely on `Scope::isThroughLoop` to detect when a variable defined outside the loop is assigned within it. In that case, `Scope::set` invokes a callback that checks the new value is a subtype of the original; this prevents widening locals beyond what preceding iterations guarantee. `Scope::merge` and `Scope::popMerge` orchestrate data-flow joins for `if`, loops, and `try-catch` constructs. Finally blocks run in dedicated scopes so that any assignments there are merged back independently of whether the try/catch completed abruptly. `TypeCheckStmt.sk` creates dedicated scopes for loops so that refinements made inside a loop body do not pollute the outer environment unless the variable is guaranteed to be initialized. For example, `for` statements execute the initializer in a non-loop scope, evaluate the condition in a loop scope (refining guard locals), execute the body, and merge changes back while respecting loop-carried dependencies. `switch` statements first ensure exhaustiveness (`switchCheckExhaustiveness`) and then evaluate each branch in its own scope, inserting synthetic equality checks when a case label is constant to benefit from `evalCond`. Branch scopes are merged, but branches lacking `break` statements are rejected (`switch` in `src/TypeCheckStmt.sk`).

8 Expression Checking Algorithms

8.1 Literal and Reference Evaluation

`TypeChecker::expr` (`src/TypeCheckExpr.sk`) is the workhorse. It handles:

- Literal numbers, bigints, strings, booleans, `null`, and `undefined`, using literal singleton constructors when `mostPrecise` is true (e.g., when binding a `kind` field).
- Template literals: each interpolation must be "stringifiable," verified via `checkStringifiable` (`src/TypeCheckUtils.sk`); the whole expression evaluates to `string`.
- Variable references: locals are retrieved from `Scope`. Otherwise, the resolver looks through builtins (resolved using `defMap`) and module-level declarations; only `const` exports are eligible for inline value typing.

The helper `asConstExpr` recognizes literal constructs, enforces that redundant `as const` annotations are rejected, and produces singleton `TType` values.

8.2 Object and Algebraic Construction

Object literals are folded into `TObject` or `TAlgebraic` depending on whether a distinguished `kind` field appears. The expression checker in `TypeCheckExpr.sk` merges spread fields directly by iterating the run-time types produced for each spread expression (`TEnv.createTFields` is reserved for type annotation normalization). Embedded method declarations (`function` fields) are still type-checked by recursively invoking `TypeChecker::funDef` with a synthetic class context. Tuple literals use `EArray` but yield `TTuple` types (reflecting the "array literals produce tuples" rule). After iterating fields, `createObject`—the helper defined in `src/TType.sk`—constructs either a `TObject` or a `TAlgebraic` value. For ADTs, each branch's field set is preserved, and the checker later relies on this structure to refine by tag. Index signatures captured through `[key: K]: V` syntax are preserved in `TFields.indexes`, ensuring that loops over dictionaries or `in` checks behave predictably.

8.3 Calls, Constructors, and Templates

Function calls go through `TypeChecker::call`, which iterates every possible overload (functions and methods) carried by the callee's `TFuns` value. For each overload, it:

1. Instantiates type parameters either from explicit `targs` or fresh `TVar`s.
2. Repeatedly checks arguments against parameters until the substitution stops evolving (`subst.version` loop).
3. Validates optional and spread parameters, including tuple element extraction for `...args`.
4. Records call graph edges via `recordCalls`, enabling later analyses in `CGNodes`.

Constructor calls (`new C(...)`) use `TEnv.newConstructor` to fetch the constructor type (`CstrType`) and then delegate to `callConstructor`. `super(...)` is only available when `TypeChecker::super` is populated (during class checking). Template strings, `await`, ternary expressions, and anonymous closures are also handled inline. `anonFunction` requires explicit parameter annotations (enforced per `skjs-rules.md`), synthesizes a `TFuns` value, and records it in the call graph. Special cases

cover intrinsic functions: `console.assert` triggers an eager `evalCond` on its first argument so that successive code benefits from the refinement. Builtins are resolved via `TypeChecker::expr` consulting `defMap` with either the user file name or the builtins file configured via the CLI; this is how DOM-like globals can be provided through `@types/skjs`.

8.4 Assignments and Local Object Growth

Additional expression forms include:

- Arithmetic and comparison operators: numeric comparisons consult `getExprPrimKind` to ensure operands share the same primitive kind, rejecting mixed-type comparisons. Logical operators require boolean operands and feed into `evalCond` for short-circuit refinement.
- The `in` operator: `TypeChecker::expr` calculates whether both true and false outcomes are possible given the operand types. If both outcomes are possible, the result is boolean; if not, a "This expression is always ..." error is raised.
- `typeof`: returns the string literal type of the operand; `TypeCheckRefine` uses the operand to narrow locals when compared against literal strings.
- `await`: ensures the awaited expression is a `Promise<T>` and yields `T`.
- Conditional expressions: evaluate both branches in isolated scopes with condition-driven refinements, then union the resulting types.

The helper `TypeChecker::get` (defined in `src/TypeCheckExpr.sk`) also deserves mention: it enforces property access rules, differentiates between methods and fields, and implements optional chaining by unioning with `undefined` when necessary.

8.5 In-Operator Semantics

Handling `key in object` expressions spans both `TypeCheckExpr.sk` and `TypeCheckRefine.sk`. First, `expr` infers the type of the LHS and RHS. For objects, it extracts candidate field maps and tracks whether the expression can evaluate to both true and false. Arrays, tuples, and ADTs are supported by over-approximating their keys via `TEnv.keys`. Based on this analysis:

1. If neither outcome is possible, the checker emits "Unexpected in" along with type witnesses for both sides.
2. If only one outcome is possible, the checker raises "This expression is always true/false," mirroring TypeScript's control-flow analysis.
3. Otherwise, the result type is boolean, and `evalCond` is called so that the RHS local is refined: on the true branch, SKJS optionally adds the field to the object's type (provided no index signature conflicts); on the false branch, it removes the field from `TFields`.

This dual-phase approach ensures the operator participates in flow analysis rather than being treated as a mere boolean. `assign` begins by validating L-values. Assignments to locals update the scope with the RHS type after ensuring the new type is compatible with the old type (via `checkSubType`). When assigning through property accesses on tracked "local objects," SKJS allows fields to grow until the object escapes. The `localObjects` map in `TCEnv` records whether a local object can still grow (`LOCanGrow`) and an optional bounding type. Once the object is observed (e.g., by reading the variable elsewhere), its state flips to `LOUsed`, freezing the field set (`src/TypeCheckExpr.sk`).

9 Statements, Functions, and Classes

9.1 Statements

`TypeChecker::stmt` (`src/TypeCheckStmt.sk`) enforces language-level constraints:

- `var` declarations are forbidden; only `let` and `const` are allowed (rule 1 in `skjs-rules.md`).
- Bindings require explicit types (with optional defaults). Tuple and object destructuring uses `bind`, which recursively matches patterns to tuple or object types, verifying shapes and optional defaults.
- Conditionals call `checkBool`, create branch scopes, and merge them respecting liveness (terminal blocks with guaranteed `return` or `throw` do not contribute to the merged scope).
- Loops establish loop scopes (`isLoop=true`) such that assignments to captured variables force subtype checks to ensure convergence.
- `try/catch/finally`: the checker rejects explicit catch-type annotations (they cannot be trusted) and uses `TType::top` for the error binding unless the user omitted a parameter.

`switch` statements ensure each `case` either ends with a terminal statement or explicitly `breaks`. A `default` branch is optional, but if the explicit cases fail to cover every reachable value the checker reports "Missing branch" diagnostics. Loop forms deserve special mention. `ForInOf` statements differentiate between key iteration (`for-in`) and value iteration (`for-of`). The former intersects the target's `keyof` approximation with `string` (`TEnv.forInKeys`); the latter asks `getValueType` to compute array/tuple element types or `[Symbol.iterator]` results. After each iteration, `Scope::popScopeReplace` merges the body scope back, ensuring that refinements (e.g., `if (item !== undefined)`) persist for subsequent iterations when sound.

9.2 Pattern Binding Walkthrough

Binding patterns are handled by `TypeChecker::bind`, which recursively visits tuple and object patterns. Consider the following snippet, inspired by destructuring in `tests/expect-error/destructuring.ts`:

```
function describePerson(person: { name: string; age?: number }): number {
  const tuple: [number, string] = [1, "a"];
  const [first, second]: [number, string] = tuple;
  const { name, age = 42 }: { name: string; age?: number } = person;
  return first + age;
}
```

During checking:

1. The tuple pattern is matched against the tuple type. Size mismatches trigger an "Invalid tuple pattern size" error quoting both the pattern and the tuple type witness.
2. The object pattern collects explicit fields and (if present) a rest binding. Each field is looked up across all branches for ADTs or all shapes for `TObject`, aggregating types when multiple shapes provide the same field.

3. Default initializers are type-checked, but SKJS warns when a default is "useless" because the field cannot be `undefined`. This is implemented by comparing the field type against `undefined` via `isSubTType`.

4. Rest bindings receive the object type with the extracted fields removed, using `TObject.removeFields`.

This walkthrough demonstrates how even seemingly simple destructuring involves DNF reasoning, optionality checks, and precise error reporting.

9.3 Function Checking

`TypeCheckFun.sk` orchestrates function declarations, including class methods. `funDefSetup` validates type parameters, binds `this`, requires explicit annotations for every parameter, and prohibits patterns or decorators that SKJS cannot model. Defaults are type-checked in a fresh scope so that the function body does not see uninitialized parameters. Async methods must return a `Promise` type with an explicit type argument; otherwise, `checkSubType` issues "Async functions must return ..." errors. Bodies must return on every execution path unless the return type is void. Interfaces cannot supply method bodies, whereas classes must (unless the method is marked abstract, which is currently unsupported per `skjs-rules.md`). Function checking proceeds through the following phases:

1. **Parameter binding**: Each parameter name is normalized (spread parameters must be last) and inserted into the scope with its declared type. Optional parameters have `undefined` unioned in automatically.

2. **Default evaluation**: Default expressions run in a temporary scope, ensuring they cannot reference later parameters and that their types conform to the parameter type.

3. **Return tracking**: `this.retType` is set before type checking the body so that nested `return` statements can validate their expressions immediately.

4. **Async validation**: When `async` modifiers are present, `TypeCheckFun.sk` checks that the annotated return type is `Promise<U>` and sets `this.isAsync` so that nested control-flow utilities know they are inside an async context.

5. **Call graph recording**: Each function is associated with a `CGNode` that captures source span and kind (free, method, constructor). Calls emanating from the function are recorded via `recordCalls`, feeding the incremental analysis pipeline described earlier.

9.4 Class Checking

`TypeCheckClass.sk` enforces:

- Single inheritance for concrete classes (interfaces can extend multiple bases). Constructor signatures in derived classes must match their parents, echoing rule 3 in `skjs-rules.md`.

- No optional type parameters on classes; each `tparam` must appear explicitly.

- Field modifiers are largely unsupported (private/protected/readonly emit TODO diagnostics), and `static` fields are forbidden.

- Constructors cannot be async, cannot have modifiers, and must not declare return types. When absent, every field without a default must be optional, otherwise "Uninitialized field" is reported.
- Implements clauses trigger subtyping checks between the synthetic `this` parameter and the interface type returned by `TEnv.new`.

During constructor checking, SKJS creates a "partial this" type that only includes fields initialized with values. The constructor body runs in this context, and upon exit `checkSubType` ensures the constructed object matches the full class type (after filtering methods so that only data fields remain). The optional tracking of `super` allows `super(...)` calls in derived classes and merges the resulting type back into `this`. Class declarations also interact with the class hierarchy recorder (`ClassHierarchy.sk`). Each time a class extends another, `recordClassHierarchy` captures the relationship so tooling (e.g., language servers) can later answer "show me subclasses" queries. Interfaces leverage the same mechanism, allowing the checker to reuse resolution logic when verifying `implements` clauses.

10 Stubs and Testing

10.1 Import Stubs and Ambient Modules

The `import-stubs/` directory mirrors the module structure the checker must resolve: Fastify (`import-stubs/fastify`), Pino, `@types`, `@fastify`, and Skip runtime adapters. Each stub file exports TypeScript declarations that SKJS parses and type-checks just like project sources, allowing real-world programs to import these modules without needing their full implementations. Specialized stubs under `import-stubs/for-tests` provide fixtures for the expect-error suite, ensuring that diagnostics tied to external modules remain stable.

10.2 Examples and Expect-Error Suites

`examples/src/` demonstrates idiomatic code: `type_refinement_examples.ts` underscores the "locals only" refinement rule, while `class_inheritance_test.ts`, `try_catch.ts`, and `test_ternary_refinements.ts` illustrate class semantics, exception handling, and ternary-based narrowing. Negative tests under `tests/expect-error` cover every rule in `skjs-rules.md`—for example, `control-flow-refinement.ts` asserts that post-guard assignments refine union types, `optionalChaining.ts` captures unsupported optional chaining forms, and `template-literal-non-string.ts` verifies template interpolation errors. These suites are executed by `tests/Main.sk`, which loads helpers from `tests/Helpers.sk`.

10.3 Conformance Harness

The repository can import upstream TypeScript conformance tests by running `devtools/copy-ts-tests.sh`. The script copies TypeScript files and their reference error baselines into `tests/ts-conformance`, ensuring that SKJS can be evaluated against real-world language edge cases. The harness enforces file layout invariants (only `.ts` files) and ensures the baseline errors accompany each copied test, enabling automated comparison. Conformance imports preserve directory structure, so SKJS can mirror TypeScript's categorization (e.g., `cases/compiler/`). When combined with the expect-error harness, maintainers can rapidly determine whether a change loosens or tightens type rules by diffing the produced diagnostics.

11 Module Reference Summary

Table 1 summarizes the most relevant modules and their responsibilities. Each entry links the conceptual role back to the concrete file paths so that future contributors can target the correct layer when extending SKJS. This reference complements the narrative sections by giving a concise

Module	Responsibility
`src/TType.sk`	Defines the core `TType` class, clause manipulation, and smart constructors for bottom/top/single types.
`src/TEnv.sk`	Normalizes AST types, instantiates classes, maintains type-parameter scopes, and exposes key/field introspection helpers used by both declarations and expressions.
`src/TypeCheckExpr.sk`	Implements expression typing, including literals, object construction, calls, property access, optional chaining, `await`, and anonymous functions.
`src/TypeCheckStmt.sk`	Checks statements, builds and merges scopes, and enforces loop/conditional semantics as well as `switch` exhaustiveness.
`src/TypeCheckClass.sk`	Validates class declarations, inheritance, constructor correctness, and field initialization rules.
`src/TypeCheckFun.sk`	Handles free functions and methods, parameter binding, async validation, and body checking.
`src/TypeCheckRefine.sk`	Performs flow-sensitive refinements for equality, `typeof`, `instanceof`, and `in` expressions.
`src/TypeCheckScope.sk`	Provides the mutable scope lattice, including binding, lookup, and merging logic with loop-awareness.
`src/TypeCheckEnv.sk`	Encapsulates the mutable checking environment, substitution, call graph context, and local-object tracking.
`docs/binary-distribution.md`	Documents CI/CD flows, artifact naming, and retrieval scripts, guiding both maintainers and consumers.

Table 1: Key SKJS modules and their core responsibilities.

"cheat sheet" for navigating the repository.

12 Surface Language Constraints and Future Work

12.1 Current Limitations

`skjs-rules.md` enumerates the supported subset and open issues. Highlights include:

- No top-level `let` (rule 1) and no `var` anywhere (`TypeCheckStmt.sk`).
- Literal array syntax creates tuples; use `new Array<T>()` for homogeneous collections (rule 2), enforced by `EArray` emitting `TTuple`.
- Constructor signatures must align across inheritance hierarchies (rule 3, `TypeCheckClass.sk`).
- Private/protected fields, abstract methods, `keyof`, function overloading, and the `any` type are unsupported (rules 4, 4b, 8, 10, 17).

- The only builtin console helpers are `log`, `assert`, and `error` (rule 5); additional helpers must be provided by user stubs.
- Only locals may be compared to `null`/`undefined`/string literals (rule 15), enforced by `TypeCheckRefine`.
- Optional chaining and template literal corner cases emit targeted diagnostics (`tests/expect-error/optionalChaining.ts`, `tests/expect-error/template-literal-non-string.ts`).
- Nested function declarations are rejected; users must declare top-level helpers or closures instead (rule 12, enforced by `TypeCheckExpr.sk`).
- Boolean literal casts such as `false as false` are not implemented (rule 19), resulting in "Construction not supported" errors originating from `ParserExpr.sk` and enforced in `TypeCheckExpr.sk`.
- The import system requires relative `./path.js` specifiers (rule 14); `ModuleResolver.sk` emits errors when encountering bare specifiers to avoid ambiguous resolution.

These restrictions, while limiting, are intentional: they produce a sound core that can be used by other Skip-based tools without modeling the entirety of the TypeScript specification.

12.2 Future Enhancements

Potential work items documented in `skjs-rules.md` and implied by TODOs in the source include:

1. Implementing full support for field modifiers and abstract members (`src/TypeCheckClass.sk`). This would involve tracking visibility in the type environment, enforcing override rules, and emitting errors when an abstract method remains unimplemented in concrete subclasses.
2. Allowing `keyof` and treating it symbolically rather than as an over-approximation (`TEnv.keyOfType`). A symbolic implementation would preserve relationships between objects and their keys, enabling the compiler to reject invalid property access without enumerating every literal.
3. Broadening refinement targets beyond locals, potentially by tracking aliasing relationships safely. This could leverage borrow-like analysis to permit comparisons on `obj.field` when the object is provably immutable within the scope.
4. Enabling function overloading resolution beyond strict arity matching, perhaps by reusing TypeScript's overload ordering rules. That would require ranking overloads and rechecking arguments according to specificity, a non-trivial change to `TypeChecker::call`.
5. Supporting template literal inference beyond stringifiable primitives, so user-defined `toString` implementations can participate. This will likely require trait-like constraints on class definitions plus a new `Stringifiable` interface in `import-stubs/@types`.
6. Improving `in`-operator precision by distinguishing guaranteed vs. optional fields instead of the current "can be true/false" heuristic (`TypeCheckExpr.sk`). Doing so would eliminate some false positives in patterns involving optional chaining plus `in`.
7. Adding native array literal support distinct from tuples, which would require rethinking `EArray` typing rules and `TEnv.keys`. Ideally SKJS would desugar `[1,2,3]` into `Array<number>` when all elements share a type and into tuples otherwise.

13 Illustrative Flow Derivation

To illustrate how SKJS combines type representation, subtyping, and refinement, consider a `Result<T>` algebraic type:

$$\text{Result}(T) = \{\text{kind} : \text{“}success\text{”}, \text{data} : T\} \mid \{\text{kind} : \text{“}error\text{”}, \text{message} : \text{string}\}.$$

The following pseudo-code mirrors how `TypeCheckStmt` and `TypeCheckRefine` validate a switch that refines on `result.kind` (`src/TypeCheckStmt.sk`, `src/TypeCheckRefine.sk`):

```
switch (result.kind) {
  case "success":
    // evalCond with target=true filters TAlgebraic branches
    const payload: string = result.data;
    break;
  case "error":
    const msg: string = result.message;
    break;
  default:
    // switchCheckExhaustiveness rejects this branch as unreachable
}
```

Internally:

1. `switchCheckExhaustiveness` constructs synthetic equality comparisons (`result.kind === "success"`) so that `evalCond` can narrow the scope.
2. `evalAlgebraicBranchEquality` filters the `TAlgebraic` branches by tag, returning a type whose `branches` map only contains the matching constructor.
3. Property reads within the branch (`result.data`) invoke `TypeChecker::dot`, which now sees a single branch and yields the precise field type.
4. After each branch, `switch` merges scopes to ensure post-switch code sees the union of the branch-wise types; here both branches are terminal, so no merge occurs.

This derivation demonstrates the tight interplay between DNF types, algebraic constructors, and scope-aware refinements.

14 Case Study: Refinement Workflow

To ground the preceding abstractions, consider `examples/src/type\_refinement\_examples.ts`. Each function demonstrates the “locals only” discipline enforced by `TypeCheckRefine.sk`:

1. **Local binding**: Before testing `obj.message`, the code assigns it to `let message = obj.message`. `bind` in `TypeCheckExpr.sk` records the local with type `string | null`.
2. **Guard evaluation**: When entering `if (message !== null)`, `evalCond` filters the local type to `string`, enabling method calls such as `message.toUpperCase()` without additional casts.
3. **Nested refinements**: Deeper nesting (e.g., `user.name`) repeats the process, highlighting that each property must be captured into a fresh local before comparison.

The expect-error counterpart in `tests/expect-error/control-flow-refinement.ts` encodes the same patterns but asserts that the checker rejects illegal refinements and accepts legal ones. For example, failing to rebind `apiPort` before comparing it to `undefined` would trigger the "Only locals can be compared ..." diagnostic, ensuring that users cannot rely on property comparisons that SKJS cannot soundly track.

Combining both files offers a full workflow: developers follow the idioms shown in `examples/`, while `tests/expect-error/` ensures the checker enforces the rules that make those idioms sound.

15 Union Normal Form Proof Overview

We summarize the ingredients used in the proof of union normalization:

- Clause-level unions (`Clause::union`) and clause-set unions (`Clauses::union`) preserve denotation while maintaining canonical form (no bottom clauses, a single top clause, unique key per quantified-variable set) — Lemmas 0.a and 0.b.
- Function "union" is a least upper bound under subtyping (domain = intersection, codomain = union) — Lemma 1.
- Algebraic unions are exact on layout-compatible branches and over-approximate via optionalization on layout divergence — Lemma 2.
- Recursive unions/intersections are handled by a monotone expansion operator whose least fixed point exists (Tarski) and is computed by the implementation (Kleene-style iteration) — Lemma 3.

Proof Prerequisites and Scope

We assume the well-formedness invariants in the Type Language section and the denotational semantics from Definition 1. The theorem below is stated relative to these assumptions and the case-by-case semantics captured by Lemmas 0.a, 0.b, 1, and 2.

Lemma 0.a (Clause union preserves denotation and normal form).

For any clauses c_1, c_2 with identical quantified-variable sets, $[\![c_1 \uplus c_2]\!] = [\![c_1]\!] \cup [\![c_2]\!]$, where $\uplus$ is `Clause::union`. Moreover, if either clause denotes $\top$, the result is the canonical top-clause; if either denotes $\bot$, it is discarded. This matches `Clause::union`'s construction and `Clauses::create`'s top-collapsing invariant.

Lemma 0.b (Clause-set union preserves denotation and normal form).

For any clause maps C_1, C_2, $[\![C_1 \sqcup C_2]\!] = [\![C_1]\!] \cup [\![C_2]\!]$, where $\sqcup$ is `Clauses::union`. Keys absent on one side are preserved verbatim; keys present on both sides are combined via Lemma 0.a. `Clauses::create` ensures a single $\top$ clause if any merged clause is top.

Lemma 1 (Function "union" is a least upper bound).

Let $f_1 : A_1 \to B_1$ and $f_2 : A_2 \to B_2$ be `TFun` overloads and let

$$f = \texttt{unionTFun}(f_1, f_2) \stackrel{\text{def}}{=} (A_1 \wedge A_2) \to (B_1 \vee B_2).$$

Then $f_1 \preceq f$ and $f_2 \preceq f$ (so $[\![f_1]\!] \subseteq [\![f]\!]$ and $[\![f_2]\!] \subseteq [\![f]\!]$); moreover, for any $g : A \to B$ with $f_1 \preceq g$ and $f_2 \preceq g$, we have $f \preceq g$. Hence f is the *least upper bound* of f_1, f_2 under the standard subtyping order (contravariant in parameters, covariant in results).

Proof. Upper bound: $f_1 \preceq f$ because $(A_1 \wedge A_2) \preceq A_1$ and $B_1 \preceq (B_1 \vee B_2)$; similarly for f_2. Leastness: from $f_1 \preceq g$ and $f_2 \preceq g$ we derive $A \preceq A_1$ and $A \preceq A_2$, hence $A \preceq (A_1 \wedge A_2)$; and $B_1 \preceq B$ and $B_2 \preceq B$, hence $(B_1 \vee B_2) \preceq B$. Therefore $f \preceq g$. □

Remark. When the pointwise meet $B_1 \wedge B_2$ exists, the greatest lower bound (GLB) — if desired — is $(A_1 \vee A_2) \to (B_1 \wedge B_2)$; in practical TypeScript encodings the codomain meet may collapse to `never` (e.g., distinct literal return types), which explains why `unionTFun` implements the LUB rather than the GLB.

Definition (Layout compatibility). Two object constructors c_1, c_2 with the same tag are *layout-compatible* if and only if:

(1) They share the *same discriminant field* and the discriminant values are *disjoint* literal sets.

(2) No *required* non-discriminant field appears in both with incompatible types.

(3) No *index signature* can capture the discriminant or collide with required fields (its domain excludes the discriminant key and its range is a supertype of all explicitly declared field types).

(4) Mutability does not enable writes in one branch that would be assumed readable in another (read-only fields may be merged covariantly; mutable fields require invariance).

When any condition fails, we say the branches are *layout-divergent*; the union operator must then perform *optionalization* (safe over-approximation).

Lemma 2 (Algebraic unions: two regimes).

Let c_1, c_2 be constructors with the same tag.

1. **Layout-compatible case.** If c_1 and c_2 have the same field-name set (modulo optionality), then $[\![\texttt{TConstr::union}(c_1, c_2)]\!] = [\![c_1]\!] \cup [\![c_2]\!]$.

2. **Layout-divergent case.** If field-name sets differ, `TConstr::union` optionalizes unmatched fields and unions matching fields. Then

$$[\![c_1]\!] \cup [\![c_2]\!] \subseteq [\![\texttt{TConstr::union}(c_1, c_2)]\!],$$

   and the inclusion can be strict because values combining both unmatched fields are admitted by the right-hand side but not by the left.

Extending tag-wise yields the analogous statement for `TAlgebraic::union`.

Proof. Case (1) follows by per-field covariance and component-wise unions. Case (2) follows because optionalization admits values whose field-sets strictly contain those of either input. □

Lemma 3 (Monotone expansion and least fixed point).

Let $\mathcal{L}$ be the complete lattice of clause-maps ordered by pointwise subset on denotations. Define $F : \mathcal{L} \to \mathcal{L}$ to be the one-step expansion operator that replaces a frontier set of instances by their clause expansions using `TType::union`/`TType::inter` on clause-maps and accumulates results via `Clauses::union`. Then:

1. F is *monotone*: $X \subseteq Y \Rightarrow F(X) \subseteq F(Y)$, because every constructor used is monotone in its arguments (Lemmas 0.a and 0.b and covariance/contravariance handled by the helpers).
2. The semantic interpretation of a (mutually) recursive type is the least fixed point μF, which exists by Tarski's theorem.
3. The implementation's memoized `expand_` computes an increasing chain $X_0 \subseteq X_1 \subseteq \ldots$ with $X_{n+1} = F(X_n)$ until it stabilizes at a post-fixpoint X^*; by monotonicity and the use of normal forms, $X^* = \mu F$.

Proof. Monotonicity is by structural induction on the constructors and Lemmas 0.a–0.b. Existence of μF follows from Tarski. The memoized expansion simulates Kleene iteration over normal forms; stabilization implies a fixed point, and minimality follows from starting at $\bot$ and monotonicity. □

Theorem (Union normal form semantics, by constructors).

For any types T_1, T_2 expressible in `TType`, let $U =$ `TType::union`(T_1, T_2) and write $[[\cdot]]$ for denotation. Then, component-wise by constructor:

- Primitives, tuples, promises, objects: $[[U]] = [[T_1]] \cup [[T_2]]$ (exact set-theoretic union).
- Functions: $[[U]] \subseteq [[T_1]] \cup [[T_2]]$, and U is the least upper bound under function subtyping (Lemma 1).
- ADTs (layout-compatible branches): $[[U]] = [[T_1]] \cup [[T_2]]$.
- ADTs (layout-divergent branches): $[[T_1]] \cup [[T_2]] \subseteq [[U]]$ (over-approximation by optionalization; Lemma 2).

In all cases, U preserves the canonical invariants (no bottom clauses, a single top clause, unique quantified-variable keys). Termination and well-definedness follow from Lemma 3.

Proof sketch. Expand both inputs using Lemma 3. For primitives, tuples, promises, and objects, `SType::union` implements the set-theoretic union directly; equality follows by the inductive hypothesis and Lemmas 0.a/0.b for clauses. For functions, apply Lemma 1. For ADTs, apply Lemma 2, splitting on layout compatibility. Clause-level joins remain sound because `Clause::union` and `Clauses::union` preserve denotation and normal form (Lemmas 0.a/0.b). Instances are combined by set union, which matches the denotation of instance carriers. □

Case analysis for `SType::union`. For completeness we record how each constructor contributes to the proof:

- **Primitives (`TPrimUnit`/`TPrimValue`).** Units are singletons, so unions defer to identity; literal families widen by uniting their value sets. This matches the semantic union because literal sets are finite and tracked explicitly.

- **Promises.** The helper produces a promise whose payload type is the union of the operands. Since promises are covariant in their payload, soundness follows immediately from the inductive hypothesis on the payload types.
- **Tuples.** Component-wise unions preserve positional variance. Length mismatches lead to unions with `undefined` for the missing tail, modelling TypeScript's widening behaviour for tuple unions.
- **Objects.** Field unions are covariant; missing fields are marked optional, preserving semantics of structural subtyping. Index signatures are joined using the same helper, yielding a safe over-approximation.
- **Functions/algebraic data types.** Covered by Lemma 1 and Lemma 2.

Coverage Checklist

Constructor/Component	Proof Location
`STypes` (union carrier)	Case analysis (by constructors), Def. 1
`TPrimUnit`, `TPrimValue`	Case analysis (Primitives)
`TTuple`	Case analysis (Tuples)
`TPromise`	Case analysis (Promises)
`TObject` (fields, index)	Case analysis (Objects)
`TAlgebraic`/`TConstr`	Lemma 2 (layout-compatible/divergent)
`TFuns`/`TFun`	Lemma 1 (LUB under subtyping)
`Clause`	Lemma 0.a (denotation + normal form)
`Clauses`	Lemma 0.b (map union + top canonicalization)
`TUnion`, `TInter` (inst placeholders)	Definition 1, Lemma 3 (lfp semantics)
`TSInst` (named types)	Definition 1 (lfp by alias equation), Lemma 3
`TVar` (type params)	Definition 1 (ρ valuation)
`Top`/`Bottom` shaping	Invariants; Lemma 0.b (top collapse), Clause omission

Table 2: Constructor coverage and where each is discharged in the proof.

Running Examples

Primitive union.

```
// Expect: ok - union is number | string
function f(x: number | string): string { return x.toString(); }
```

Tuple normalization.

```
// Expect: ok - trailing element widens to number | undefined
type Pair = [number, string] | [number, undefined];
const pair: Pair = [1, undefined];
```

Promise covariance.

```
// Expect: ok - Promise payload unions to number | string
async function fetchValue(flag: boolean): Promise<number | string> {
  return flag ? 42 : "fallback";
}
```

Object optionalization.

```
// Expect: ok - optional field present only on one branch
type Config =
  | { kind: "file"; path: string }
  | { kind: "http"; url: string };
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const cfg: Config = { kind: "http", url: "https://example.com" };
```

Function LUB (overload merge, not behaviour union).

```
// Two separate arrows; the merged (LUB) arrow has domain = number & string (never),
// and return = "n" | "s".
type F1 = (x: number) => "n";
type F2 = (x: string) => "s";
// Merged LUB (overload-merge view):
type LUB = (x: number & string) => "n" | "s"; // effectively uncallable; safe upper bound

// Overloads remain the recommended user-facing form:
type Read = ((x: number) => "n") & ((x: string) => "s");
declare const r: Read;
const a: "n" = r(1);       // Literal return preserved
const b: "s" = r("one"); // Literal return preserved
```

ADT layout-divergent union.

```
// Expect: union admits optionalized fields; behaviour may be over-approximated
type A = { kind: "a"; a: number };
type B = { kind: "b"; b: string };
type U = A | B;
```

Recursive type expansion.

```
// Expect: Tree type resolved via least fixed point
type Tree = { kind: "leaf" } | { kind: "node"; left: Tree; right: Tree };
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const leaf: Tree = { kind: "leaf" };
```

Implementation Cross-Reference (Union/Intersection/Normalization)

Formal Proof

Definition 1 (Semantic interpretation with environments). Let $\mathcal{V}$ be the set of runtime values modelled by SKJS. Let ρ be a valuation mapping type parameters to subsets of $\mathcal{V}$. The denotation function $[\![\cdot]\!]_\rho$ is defined mutually as follows:

$$[\![\mathrm{TType}(I, C)]\!]_\rho = \left(\bigcup_{i \in I} [\![i]\!]_\rho\right) \cup \left(\bigcup_{c \in C} [\![c]\!]_\rho\right)$$

$$[\![\mathrm{Clause}(\vec{X}, S)]\!]_\rho = \left(\bigcap_{X \in \vec{X}} \rho(X)\right) \cap \begin{cases} [\![S]\!]_\rho & \text{if } S \text{ present} \\ \mathcal{V} & \text{otherwise} \end{cases}$$

$$[\![S_1 \mid \cdots \mid S_n]\!]_\rho = \bigcup_{i=1}^{n} [\![S_i]\!]_\rho \qquad \text{for } S\text{-types}$$

Construct	Implementation sites (files / helpers)
`TType` (union)	`src/TTypeUnion.sk`: `TType::union`, `TType::union_`, `unexpandedUnion`
`TType` (inter)	`src/TTypeInter.sk`: `TType::inter`, `TType::inter_`, `unexpandedInter`
`STypes` carrier	`src/TTypeUnion.sk`: `STypes::union`; `src/TTypeInter.sk`: `STypes::inter`
`Clause`/`Clauses`	`src/TTypeUnion.sk`: `Clause::union`, `Clauses::union`; `src/TTypeInter.sk`: `Clause::inter`, `Clauses::inter`
`Clauses normalization`	`src/TType.sk`: `Clauses::create`, top canonicalization; `TType::create` (instances stripping on top)
`TInsts`/placeholders	`src/TTypeUnion.sk`: `TInsts::union`, `class TUnion`; `src/TTypeInter.sk`: `class TInter`
`TPrimUnit`	`src/TTypeUnion.sk`: `unionSType` (primitive-unit branch); `src/TTypeInter.sk`: `interSType` (kind equality)
`TPrimValue`	`src/TTypeUnion.sk`: `TPrimValue::union`; `src/TTypeInter.sk`: `TPrimValue::inter`
`TPromise`	`src/TTypeUnion.sk`: `unionSType` (`TPromise`); `src/TTypeInter.sk`: `interSType` (`TPromise`)
`TTuple`	`src/TTypeUnion.sk`: `unionTTuple`; `src/TTypeInter.sk`: `interTTuple`
`TObject`/`TFields`	`src/TTypeUnion.sk`: `unionTObject`, `unionTFields`, `TField::union`, `TIndexField::union`; `src/TTypeInter.sk`: `interTObject`, `interTFields`, `TField::inter`, `TIndexField::inter`
`TAlgebraic`/`TConstr`	`src/TTypeUnion.sk`: `TAlgebraic::union`, `TConstr::union`; `src/TTypeInter.sk`: `TAlgebraic::inter`, `TConstr::inter`
`TFuns`/`TFun`	`src/TTypeUnion.sk`: `TFuns::union`, `unionTFun`; `src/TTypeInter.sk`: `TFuns::inter`, `interTFun`

Table 3: Where each union/intersection case is implemented in the codebase.

and the remaining constructors (`TPrimValue`, `TTuple`, `TFuns`, `TAlgebraic`, etc.) use their standard set-theoretic interpretation parameterized by ρ:

- $[\![\mathrm{TVar}(X)]\!]_\rho = \rho(X)$.
- $[\![\mathrm{TSInst}(N, \vec{T})]\!]_\rho$ is defined by the (possibly recursive) type alias equation for N applied to arguments $[\![\vec{T}]\!]_\rho$, interpreted as the least fixed point (see Lemma 3).
- $[\![\mathrm{TUnion}(T_1, T_2)]\!]_\rho = [\![T_1]\!]_\rho \cup [\![T_2]\!]_\rho$; $[\![\mathrm{TInter}(T_1, T_2)]\!]_\rho = [\![T_1]\!]_\rho \cap [\![T_2]\!]_\rho$.
- $[\![\mathrm{TTuple}(T_1, \ldots, T_n)]\!]_\rho = [\![T_1]\!]_\rho \times \cdots \times [\![T_n]\!]_\rho$.
- $[\![\mathrm{TFuns}]\!]_\rho$ is the set of closures whose call behaviour matches the parameter/return constraints under ρ (contravariant in parameters, covariant in returns).
- $[\![\mathrm{TAlgebraic}]\!]_\rho$ is the tagged union of its branches, each branch a record with the tag field fixed and remaining fields interpreted covariantly.

16 Algebra of Unions (laws used in code)

We list the small algebra actually used by `TTypeUnion`, `Clause::union`, and `TFuns::union`:

- Idempotence, commutativity, associativity of | and $\wedge$ on clauses with equal quantifier keys.
- Absorption with $\bot, \top$: $T \mid \bot \equiv T$, $T \wedge \top \equiv T$, $T \mid \top \equiv \top$, $T \wedge \bot \equiv \bot$.
- Distributivity used in normalization: intersections distribute over unions inside a clause; cross-clause distribution requires matching α-normalized quantifier keys.
- Functions: $(A_1 \to B_1) \sqcup_{\to} (A_2 \to B_2) = (A_1 \wedge A_2) \to (B_1 \vee B_2)$ is the *LUB*. The GLB, when defined, is $(A_1 \vee A_2) \to (B_1 \wedge B_2)$.
- Algebraic (tagged) unions: exact when branches are layout-compatible (Def. 15); otherwise we optionalize unmatched fields and union-match shared fields (safe over-approximation).

Tempting but *invalid* rewrites (explicitly avoided). To prevent classic unsoundnesses, the implementation *does not* apply the following equalities:

- $(A \to C) \wedge (B \to D) \not\equiv (A \vee B) \to (C \wedge D)$ (overload collapse; see the examples below).
- $(A \to C) \vee (B \to C) \not\equiv (A \vee B) \to C$ (either function $\neq$ one function handling both domains).
- $(A \vee B) \to C \not\leq A \to C$ (domains are *contra*variant, not covariant).
- $(F_1 \wedge F_2) \vee (G_1 \wedge G_2) \not\equiv (F_1 \vee G_1) \wedge (F_2 \vee G_2)$ (unsafe factorization).
- $(\forall X.\, S_1) \vee (\forall X.\, S_2) \not\equiv \forall X.\, (S_1 \vee S_2)$ (monomorphic vs. polymorphic confusion).

All five appear in the *Tricky cases and litmus tests* below as short, in-paper TypeScript snippets that demonstrate why these rewrites are disallowed in our normalizer and subtyping.

```
// (A -> C) /\ (B -> D)  vs. (A \_/ B) -> (C /\ D)
type Left = { tag: "L"; l: number };
type Right = { tag: "R"; r: number };
declare const f: ((s: string) => Left) & ((n: number) => Right);
// EXPECT-ERROR: Missing type annotation
const value = f(Math.random() < 0.5 ? 0 : "s");
// const both: Left & Right = value; // invalid: a single call cannot return both branches

// (A -> C) \_/ (B -> C)  vs. (A \_/ B) -> C
declare const either: ((s: string) => number) | ((n: number) => number);
// const merged: (x: string | number) => number = either; // invalid

// (F1 /\ F2) \_/ (G1 /\ G2)  != (F1 \_/ G1) /\ (F2 \_/ G2)
type F = ((s: string) => Left)  & ((n: number) => Right);
type G = ((s: string) => Right) & ((n: number) => Left);
declare const pick: F | G;
// const needBoth: (z: Left & Right) => void = pick; // invalid

// Control-flow merges use set union of behaviours, not the function LUB.
declare const coin: boolean;
// EXPECT-ERROR: Missing type annotation
const g = coin ? (): "L" => "L") : (): "R" => "R");
```

```
// EXPECT-ERROR: Missing type annotation
const v = g(); // type is "L" | "R"
```

17 Expressivity, Promise Semantics, and Framework Interoperability

Despite the conservative semantics outlined above, some idioms that are commonplace in everyday TypeScript do not type-check in SKJS yet. The gap stems chiefly from the areas where the checker deliberately under-approximates in order to stay sound. We highlight the most visible instance—lack of promise flattening—and then list other notable differences. We then revisit function-parameter variance and show how to interoperate safely with popular back-end frameworks that rely on TypeScript's more permissive (bivariant) method checking.

Missing Promise Flattening. JavaScript (and therefore TypeScript) treats promises monadically: any value returned from an `async` function is wrapped in a single layer of `Promise`, and `await` unwraps repeatedly until a non-promise value is produced. SKJS models `Promise<T>` as the covariant constructor `TPromise(T)`, but it does not flatten nested payloads. As a result, programs that rely on nested promises type-check in TypeScript but are rejected here.

```
declare function toastId(): string;
declare function loadMessage(id: string): Promise<string>;

declare const waitForToast: Promise<Promise<string>>;

declare function showToast(msg: string): void;

async function showWelcome(): Promise<void> {
  // Both runtimes eventually obtain a string, but SKJS sees Promise<string>.
  const message = await waitForToast;
  // EXPECT-ERROR: Invalid call
  // EXPECT-ERROR: This is a Promise
  // EXPECT-ERROR: It is incompatible with: string
  showToast(message);              // TS: ok  SKJS: rejects
}

async function reload(): Promise<void> {
  const id = toastId();
  const text = await loadMessage(id);
  showToast(text);                 // both checkers accept
}
```

Other Notable Limitations (worked out). For each item we state whether it is worth including as an expressivity gap, then we give a realistic example and discuss workable mitigations.

Function unions. Include? *Yes* (moderate). Why it matters. In practice, teams sometimes want to hold a heterogenous callback in one slot (e.g., handler can be either "simple" or "contextual"). In SKJS a value of type $(A \to R) \vee (B \to R)$ is *not* callable unless A and B are identical; this avoids the classic "overload collapse" unsoundness and matches strict TypeScript behaviour.

Example (middleware-like handlers).

```
type Req = { id: string };
type Ctx = { traceId: string };
type Res = { ok: true };

type H1 = (req: Req) => Promise<Res>;
type H2 = (req: Req, ctx: Ctx) => Promise<Res>;

declare const h: H1 | H2;

// EXPECT-ERROR: Missing return type
async function run(req: Req, ctx: Ctx) {
  // You do not know which domain you have; both TS (strict) and SKJS reject:
  await h(req, ctx); // not callable
}
```

Workarounds. Normalize to a *single* callable shape and adapt at the boundary:

```
// Normalize to a single signature with an optional arg.
type H = (req: Req, ctx?: Ctx) => Promise<Res>;

// EXPECT-ERROR: Type H1 not found
function asH(f: H1 | H2): H {
  return (req, ctx) => (f.length < 2 ? (f as H1)(req) : (f as H2)(req, ctx!));
}
```

This pattern is straight-line and cheap; it is how ecosystem libraries hide overloads internally. Alternative designs (e.g., storing tagged unions of functions) work but add boilerplate. We recommend *not* exposing function unions in public APIs; expose a single normalized signature.

Additional sanity checks. The union machinery deliberately forms the least upper bound: domains intersect, returns union. The following snippet shows that the LUB is well typed whereas the would-be GLB collapses:

```
type F1 = (x: number) => 1;
type F2 = (x: string) => 2;

// LUB is the intersection of domains and union of codomains.
type LUB = (x: number & string) => 1 | 2; // effectively never callable

// GLB would require a union domain and intersection of codomains;
// when the codomain meet is empty, the type collapses to never.
type GLB = (x: number | string) => 1 & 2; // reduces to never

// Overlap + specificity (safe overload intersection).
type Over = ((x: number) => "n") & ((x: number | string) => "ns");
declare const over: Over;
const a: "n" = over(1);
const b: "ns" = over("hi");
```

Overload resolution also obeys the usual rules: the overlap of two call signatures remains callable at the most precise domain. If we try to merge genuinely distinct domains, SKJS forces us to acknowledge the uncertainty:

```
// EXPECT-ERROR: Algebraic field kind was expected to be a constant string
```

```
// EXPECT-ERROR: Got 'a' | 'b' instead.
type O =
  | ((x: { kind: "a"; a: number }) => "A")
  | ((x: { kind: "a" | "b"; a?: number; b?: string }) => "AB");

declare const o: O;
const rA: "A" = o({ kind: "a", a: 1 });
const rAB: "AB" = o({ kind: "b", b: "" });
```

Layout-divergent algebraic unions. Include? *Yes* (moderate, correlated-field precision). Why it matters. Real services return discriminated unions such as {`kind: "ok", value: T`} | {`kind: "err", msg: string`}. SKJS safely over-approximates object unions by matching shared fields and optionalizing unmatched ones; it does *not* track correlation between the discriminator and extra fields. This keeps subtyping simple but loses precision inside branches.
Example (correlated field).

```
type Ok = { kind: "ok", value: { name: string } };
type Err = { kind: "err", msg: string };
type Result = Ok | Err;

declare const r: Result;

if (r.kind === "ok") {
  // EXPECT-ERROR: Expected }
  r.value.name;   // TS: ok (discriminated narrowing).  SKJS: may reject without a cast.
}
```

Workarounds. Supply an explicit user-defined guard and rebind a narrowed local, or project using a helper:

```
function isOk(x: Result): x is Ok { return (x as any).kind === "ok"; }

if (isOk(r)) {
  // EXPECT-ERROR: Expected }
  const ok = r as Ok;      // or rely on the guard's predicate type
  ok.value.name;
}
```

In router/middleware code, prefer threading a refined *local* (e.g., `const req2 = req as Req & { user: U }`) after each guard. The cost is low and explicit; it keeps the global type for `req` conservative while allowing local precision where needed.
Exact vs. layout-divergent branches.

```
type S = { kind: "s"; v: string };
type T = { kind: "t"; v: string };
type ST = S | T;
function format(x: ST): string {
  if (x.kind === "s") return x.v.toUpperCase();
  return x.v.toLowerCase();
}

type A = { kind: "a"; x: number };
type B = { kind: "b"; x: string };
```

```
type U = A | B;
// Any merged shape must not admit { kind: "a", x: "oops" }.
```

Index signatures with discriminants. When record types mix index signatures and explicit discriminants we retain the discriminator verbatim and only optionalize fields proven to be shared:

```
type C = { kind: "c"; [k: string]: number };
type D = { kind: "d"; flag: boolean };
type V = C | D;
function handle(v: V): void {
  if (v.kind === "d") {
    const flag: boolean = v.flag;
    // EXPECT-ERROR: Unbound name
    void flag;
  }
}
```

Mutable class overrides. Include? *Yes* (small but visible for OO-heavy code). Why it matters. TypeScript treats method parameters bivariantly; SKJS keeps the mathematically sound rule: parameters are contravariant, results covariant. As a result, some overrides that compile in TS are rejected in SKJS.
Example (event handler override).

```
class Base {
  // EXPECT-ERROR: Type Event not found
  handle(e: Event): void {}
}
class Clicks extends Base {
  // EXPECT-ERROR: Type Event not found
  handle(e: MouseEvent): void {} // TS: allowed; SKJS: reject (parameter narrowed)
}
```

Workarounds. Make the variance explicit via generics or adapter methods:

```
// EXPECT-ERROR: Optional type parameters are not supported for classes
class Base<E extends Event = Event> {
  handle(e: E): void {}
}
class Clicks extends Base<MouseEvent> {
  handle(e: MouseEvent): void {}
}
// or keep Base.handle widest and forward from a refined overload:
class SafeClicks extends Base {
  handle(e: Event) { if (e instanceof MouseEvent) this.onClick(e); }
  onClick(e: MouseEvent) {}
}
```

In framework code, prefer binding free functions (which are naturally contravariant) rather than overriding methods with narrower parameters.

Built-in libraries & ambient declarations. Include? *Yes* (pragmatic). Why it matters. Only a curated subset of `lib.d.ts` and Node typings is modelled. Many projects call

DOM APIs, `fetch`, or Node streams; unmodelled areas surface as "unknown" or missing types rather than logic errors.

Workarounds. Add slim, purpose-built declaration stubs or wrap the API behind a small, typed facade. Two practical patterns:

```
// (1) Minimal stub to satisfy the checker where full modelling is overkill.
declare function fetch(u: string, init?: unknown): Promise<{ ok: boolean }>;

// (2) Local facade with precise, app-specific types.
export interface Http {
  getJson<T>(u: string): Promise<T>;
}
```

Maintaining facades pays off quickly; it keeps the trusted surface area small and makes tests easier to write.

Two non-gaps to document (not expressivity issues). Include? *No*, but keep as guidance. Generic wrappers and `never`. When a combinator insists on a single input type for two negatively-positioned arguments, principal type inference yields `never`. This is expected and safe; see the example in Section 17.8. The ergonomic workaround is to normalize domains (e.g., one optional parameter) before passing functions to such combinators.

Branch-based refinement after `never`. Once contravariant inference has collapsed a parameter to `never`, no amount of local control-flow refinement can reintroduce information. This is by design and prevents masked unsoundness. Prefer earlier normalization or avoid unifying incompatible domains through a single type variable.

Bridging these gaps would require enhancing the type representation (e.g., monadic flattening for `Promise`, proper union-of-overloads ranking) while maintaining the invariants on which the soundness argument rests. We leave these improvements to future work.

17.1 Bivariance in TypeScript Frameworks

This section examines how SKJS's *strict contravariance for function parameters* interacts with widely-used TypeScript back-end frameworks whose APIs may exhibit *bivariance* (by design or for historical reasons). We summarize concrete handler signatures from popular frameworks, identify where bivariance actually appears, and provide *binding patterns*—without changing the type system—that make interop precise and practical.

17.2 Background: where bivariance comes from in TS

Under `--strictFunctionTypes`, TypeScript checks *function* parameter types contravariantly. However, for historical compatibility, parameters of *methods* (and constructors) are checked bivariantly.[1] This can permit unsound assignments when callbacks are expressed as *methods on object types*.

```
// TS method-parameter bivariance (illustrative unsoundness)
// EXPECT-ERROR: Forbidden field name 'kind' in a class
interface Animal { kind: "cat" | "dog" }
interface Dog extends Animal { kind: "dog"; bark(): void }

interface Sink { handle(x: Animal): void }
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
```

[1] https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-6.html

```ts
const s: Sink = {
  // Allowed in TS because 'handle' is a *method* (checked bivariantly)
  handle(x: Dog) { x.bark(); } // narrower parameter accepted
};

const someAnimal: Animal = { kind: "cat" } as Animal;
// s.handle(someAnimal); // would be unsafe at runtime
```

SKJS keeps the mathematical rule (contravariance for arrows). Interfacing with APIs that *rely* on method bivariance therefore needs care at binding boundaries.

17.3 Survey of common Node/TS back-end APIs (actual code)

Express 4/5. Express request handlers are declared as a *function type alias* (not a method), which means standard contravariant checking applies under `--strictFunctionTypes`. A representative shape is:[2]

```ts
// EXPECT-ERROR: Optional type parameters are not supported for classes
export interface RequestHandler<
  P = core.ParamsDictionary,
  ResBody = any,
  ReqBody = any,
  ReqQuery = core.ParsedQs,
  Locals extends Record<string, any> = Record<string, any>
> {
  (req: Request<P, ResBody, ReqBody, ReqQuery, Locals>,
   res: Response<ResBody, Locals>,
   next: NextFunction): void;
}
```

Assessment. There is no intrinsic reliance on bivariance in the core handler type; SKJS's contravariance matches this. In practice, friction typically comes from *declaration merging* of `express.Request` to attach fields (e.g. `req.user`) and from overload selection in Express 5 (e.g. error middleware arity; see https://stackoverflow.com/questions/71243903/accessing-middleware-injected-data-in-typescript and https://www.reddit.com/r/node/comments/1gakl3x/how_do_i_resolve_the_below_error_on_express_50/).

Koa 2.x (+ @types/koa). Koa middlewares are function types via `koa-compose` (see https://app.unpkg.com/@types/koa@2.13.8/files/index.d.ts for the definition `type Middleware<StateT, ContextT, R = any> = compose.Middleware<ParameterizedContext<StateT, ContextT, R>>`). The `Context` is frequently augmented by middleware (sessions, passport, etc.), and there are known type-coherence issues when multiple packages widen the context differently (e.g. router + session: https://github.com/koajs/router/issues/39; DefinitelyTyped discussions: https://github.com/DefinitelyTyped/DefinitelyTyped/issues/36161, https://github.com/DefinitelyTyped/DefinitelyTyped/issues/22568). These are not variance problems per se; they are ambient-augmentation and unification problems.

[2] Excerpt of `RequestHandler` from published d.ts builds: https://cdn.jsdelivr.net/npm/@umijs/deps@3.4.21/compiled/express/express-serve-static-core/index.d.ts.

Fastify 4/5. Fastify exposes route handlers as function type aliases with rich generics (Body/Query/Params/Reply), e.g. `RouteHandlerMethod<...>`, and documents how to thread types through routes and hooks (https://fastify.io/docs/latest/Reference/TypeScript/, https://fastify.io/docs/latest/Reference/Routes/). Community issues often concern handler generic constraints rather than variance (e.g. extending return types: https://github.com/fastify/fastify/issues/4685; handler incompatibility: https://stackoverflow.com/questions/72620384/generics-issue-while-using-fastify-with-typescript).

NestJS (controllers). Nest routes are *class methods* decorated with `@Get()` etc.[3] Because these are *methods*, TS's special-case bivariance can mask unsound parameter narrows when relating controller types. In practice Nest relies on runtime pipes/validation to enforce DTO shapes; at the type level, the safest binding for SKJS is to keep domain handlers as arrow functions and let controller methods forward to them (see below).

17.4 Will bivariance in frameworks block interop?

- **Express & Koa:** No fundamental blocker. Public handler signatures are function types, so SKJS's contravariant rule aligns. The main challenge is *ambient augmentation* of request/context across middleware stacks.
- **Fastify:** Also fine on variance; the complexity is in generics & hook-driven refinements. Interop hinges on where the SKJS checker observes the widened request shape.
- **NestJS:** Potential sharp edges due to method-parameter bivariance in TS. SKJS will (correctly) *reject* narrows that TS would allow on methods; prefer controller bindings that keep domain logic as arrow functions.

17.5 Binding patterns (no type-system changes)

We catalog concrete, copy-pasteable binding adapters that preserve SKJS soundness while fitting each framework's API.

Express: widen-at-boundary adapter.

```
// SKIP-CHECK
// EXPECT-ERROR: Could not find module[4]: express
// EXPECT-ERROR: Type any not found
import type { Request, Response, NextFunction, RequestHandler } from "express";

// Accept a precise, domain-level handler and adapt it to Express's RequestHandler.
export function toExpress<P, ResBody, ReqBody, ReqQuery, Locals extends Record<string, any> = {}>(
  h: (req: Request<P, ResBody, ReqBody, ReqQuery, Locals>) => Promise<ResBody>
): RequestHandler<P, ResBody, ReqBody, ReqQuery, Locals> {
  return (req, res, next) => {
    h(req).then((body) => { res.json(body as ResBody); }, next);
  };
}

// Usage (no bivariance needed):
type User = { id: string; email: string };
const getMe = async (req: Request<{id: string}, User, unknown, unknown>) => ({ id: req.params.id, email
```

[3] https://docs.nestjs.com/controllers.

```
// EXPECT-ERROR: Type app not found
app.get("/users/:id", toExpress(getMe));
```

Koa: context-guarding adapter with coherent augmentation.

```
// SKIP-CHECK
// EXPECT-ERROR: Could not find module[4]: koa
import type Koa from "koa";

// 1) Augment DefaultContext once, in one module (declaration merging).
declare module "koa" {
  interface DefaultContext {
    user?: { id: string; role: "admin" | "user" };
  }
}

// 2) Adapter: accept a narrower ctx (with guaranteed 'user') and expose a broad Koa Middleware.
type AuthedCtx = Koa.ParameterizedContext<Koa.DefaultState, Koa.DefaultContext & { user: { id: string; r

// EXPECT-ERROR: Missing type annotation
export const requireUser =
  (h: (ctx: AuthedCtx) => Promise<void>): Koa.Middleware =>
  async (ctx, next) => {
    if (!ctx.user) ctx.throw(401);
    await h(ctx as AuthedCtx);   // single, local cast after runtime guard
    return next();
  };

// Usage:
// EXPECT-ERROR: Type app not found
app.use(requireUser(async (ctx) => {
  ctx.body = { hello: ctx.user.id };
}));
```

This pattern centralizes the (sound) run-time check and limits `as`-casts to the boundary, avoiding pervasive bivariant assumptions.[4]

Fastify: preHandler-based refinement carried into the route.

```
// EXPECT-ERROR: Expected )
import type { FastifyInstance, FastifyRequest, FastifyReply, RouteShorthandOptions } from "fastify";

declare module "fastify" {
  interface FastifyRequest {
    user?: { id: string };
  }
}

// A preHandler that refines request.user at runtime:
async function auth(req: FastifyRequest, reply: FastifyReply) {
  const token = req.headers.authorization;
  if (!token) return reply.code(401).send({ error: "unauthorized" });
  req.user = { id: "u1" };
```

[4] Context coherence issues across Koa ecosystem packages: https://github.com/koajs/router/issues/39, https://github.com/DefinitelyTyped/DefinitelyTyped/issues/36161.

```
}

export function registerUserRoutes(f: FastifyInstance) {
  f.get<{ Reply: { id: string } }>(
    "/me",
    { preHandler: auth } satisfies RouteShorthandOptions, // keep types aligned
    async (req, reply) => {
      // Local refinement (sound): we just ran 'auth', so 'user' exists
      const user = req.user as { id: string };
      return reply.send({ id: user.id });
    }
  );
}
```

The key is to *widen at the boundary* (request may or may not have `user`) and *refine locally* after a checked hook. This avoids relying on bivariant handler assignment while staying faithful to Fastify's function-typed handlers.

NestJS: arrow-function controllers via adapters.

```
// Keep domain handlers as arrow functions (contravariant-safe), then wrap in Nest.
type GetUser = (id: string) => Promise<{ id: string }>;
const getUser: GetUser = async (id) => ({ id });

// Controller binds by extracting from the Nest ExecutionContext and calling the function.
import { Controller, Get, Param } from "@nestjs/common";

@Controller("users")
// EXPECT-ERROR: Unknown character: @ (64)
export class UsersController {
  constructor(/* inject service with 'getUser' */) {}
  @Get(":id")
  async get(@Param("id") id: string) { return getUser(id); }
}
```

Here the domain-level callable retains contravariant arrow semantics; the Nest method simply forwards parameters.[5] Avoid exposing method-typed callbacks to third-party code (where TS bivariance could be observed).

17.6 How important are the expressivity gaps?

- In **Express**, the gap is negligible: handlers are function types; SKJS interop is natural. Declare-merge `Request` once and prefer adapters to thread refined data.
- In **Koa**, the main issue is *context coherence*, not variance. The adapter above and one-time declaration merging are sufficient. Use explicit runtime guards where middleware promises invariants.
- In **Fastify**, the gap shows up when relying on hooks to guarantee fields—carry the refinement locally (or via helper generics) rather than expecting global type propagation.

[5] Nest controllers: https://docs.nestjs.com/controllers.

- In **NestJS**, the only material difference comes from TS method bivariance; prefer binding controllers to domain arrow functions or keep DTO parameters *wider or equal* to what the framework supplies (let pipes/validators narrow at runtime).

17.7 Additional TypeScript testlets (bivariance and bindings)

```ts
// SKIP-CHECK
// (1) Bivariance pitfall vs safe arrow functions
interface Service { call(x: { a: number } | { b: string }): void }
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const bad: Service = { call(x: { a: number }) { x.a.toFixed() } }; // allowed (method bivariance)
// Prefer arrows (checked contravariantly):
type Fn = (x: { a: number } | { b: string }) => void;
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const good: Fn = (x: { a: number }) => {}; // ERROR in strict TS (good!)

// (2) Express binding sanity
type Domain = (req: { params: { id: string } }) => Promise<{ id: string }>;
// EXPECT-ERROR: Type any not found
declare const toExpress: <P,Res,Body,Query,L>(h: any) => any;
// EXPECT-ERROR: Type any not found
declare const app: { get: (p: string, h: any) => void };
// EXPECT-ERROR: Invalid type for constant, only primitive types (not objects) are allowed
const d: Domain = async (r) => ({ id: r.params.id });
app.get("/u/:id", toExpress(d)); // OK

// (3) Koa context coherence
// EXPECT-ERROR: Type any not found
declare const requireUser: (h: (ctx: { user: { id: string } }) => Promise<void>) => any;
// EXPECT-ERROR: Unbound name
requireUser(async (ctx) => { ctx.user.id; }); // OK

// (4) Fastify hook refinement
declare const auth: (req: any, reply: any) => Promise<void>;
declare const f: { get: (p: string, o: { preHandler: typeof auth }, h: (req: any, reply: any) => any) =>
f.get("/me", { preHandler: auth }, async (req, reply) => {
  const id = (req.user as { id: string }).id; // local refine after preHandler
  reply.send({ id });
});
```

17.8 Conclusions & examples to keep

Across all our experiments we did not find a true counterexample to soundness. Every "suspicious" program either failed to type-check or collapsed its negative position to `never`. The checker is often conservative, but it is sound and predictably so.

Key takeaways (codified).

- Function unions are callable only when their domains are identical; this is by design.
- Generic combinators that require a single negative type argument for two incompatible domains infer `never`, preventing bad calls.

- Class overriding respects contravariance (no bivariant parameter narrowing); use generics or adapters instead.
- Branch-based refinement does not "resurrect" a parameter that has already been inferred as `never` through contravariant inference; this is safe but imprecise.

Minimal, self-checking snippets (good regression tests).

```
// SKIP-CHECK
// (1) Function union: not callable unless domains match.
type F = ((x: number) => string) | ((x: string) => string);
declare const f: F;
f(1);      // ERROR (both TS strict and SKJS)

// (2) Generic wrapper that forces a single domain -> never-inference.
const both = <A>(p: (a: A) => void, q: (a: A) => void) => (a: A) => { p(a); q(a); };
declare const h: ((x: number) => void) | ((x: string) => void);
const combo = both(h, h);
// combo has type (a: never) => void; calling it is impossible.
// @expect-error
combo(42);

// (3) Safe overriding uses generics (no parameter narrowing).
class B<E extends Event = Event> { handle(e: E): void {} }
class C extends B<MouseEvent> { handle(e: MouseEvent): void {} }

// (4) Refinement cannot recover from never-inference.
const g = both(h, h);
g as (a: never) => void;  // document the collapsed domain explicitly
```

These fragments mirror the rules stated earlier and serve as executable documentation. Keeping them as *expect-error* tests ensures we do not accidentally reintroduce unsafe rewrites (e.g., overload collapse) in future refactorings.

18 Testing and Behavioral Evidence

Evidence that the checker enforces the documented rules comes from:

- **Examples**: `examples/src/type\_refinement\_examples.ts` shows that declaring locals before comparison allows refinement, matching the "locals only" rule enforced in `TypeCheckRefine.sk`. Files such as `examples/src/class\_inheritance\_test.ts` and `examples/src/try\_catch.ts` exercise extends/implements checks and exception typing.
- **Expect-Error Tests**: `tests/expect-error/control-flow-refinement.ts` ensures that SKJS refines after `if` blocks; `tests/expect-error/keyof.ts` captures the "TODO: keyof" diagnostic; `tests/expect-error/void.ts` enforces void-only positions; `tests/expect-error/try-catch.ts` guards against typed catches. Each test names the specific diagnostic to assert stability.
- **Conformance Imports**: By copying TypeScript's conformance suites, SKJS is regularly stress-tested against upstream regression cases using `devtools/copy-ts-tests.sh`.

Together, these sources provide both positive assurance (examples compile) and negative assurance (expect-error tests fail as intended). The test runner (`tests/Main.sk`) loads each TypeScript file,

feeds it through the checker, and compares the resulting diagnostics to golden outputs stored alongside the tests. `tests/Helpers.sk` contains utilities for reading files, normalizing paths, and reporting diffs. Expect-error cases intentionally look for specific substrings, giving maintainers confidence that error messages stay stable even as internal representations evolve.

19 Extensions

This section is not a roadmap; it explores possible extensions and their consequences, in case they turn out to be necessary.

19.1 Per-ADT Discriminator Names

Language design. Rather than fixing a single discriminator field name (e.g., `kind`) for all algebraic data types (ADTs), each ADT may choose its own discriminator. An object/type is recognized as an ADT branch when it contains exactly one non-optional field whose type is a single string literal; that field becomes the discriminator for that ADT. Programs may mix ADTs using `kind`, `tag`, `type`, etc., in the same scope. Equality-based refinement triggers only when comparing a local's discriminator field to a string literal; if a union contains multiple ADTs with different discriminators, the refinement applies only to the matching ones. Ambiguity (multiple candidates) or optional/non-literal discriminators are rejected with precise diagnostics.

Implementation considerations. The discriminator name is stored inside the type, not as a global constant. We extend `TAlgebraic` (and branch `TConstr`) to carry a *discriminatorName*. During construction (both from object literals and normalized type declarations), the checker detects the discriminator (single literal string field), excludes it from normal object fields, and records both the tag value and the field name. To avoid destructive merging, `STypes` bucketing incorporates `discriminatorName`, allowing multiple ADTs with different discriminators to coexist in unions; union/intersection/subtyping then merge branches only when discriminator names match. Property selection and equality-based refinements consult the stored `discriminatorName`, and diagnostics quote the actual field name found in the program.